

LawLine AI

A retrieval-augmented legal assistant over Indian Supreme Court criminal-law judgments and the IPC / BNS statute text.

Metric	Value	Metric	Value
Judgments	3,000	Statute sections	878 (523 IPC + 355 BNS)
Indexed passages	177,830	Embedding model	BAAI/bge-m3, 1024-d
Corpus characters	139.3 million	Vector store	ChromaDB (HNSW, cosine)
Years covered	77 of 77 (1950-2026, complete)	Backend	FastAPI + Uvicorn
Source PDFs retained	2,740, SHA-256 verified	Frontend	React 18 + Vite + Tailwind
Candidates examined	9,350 (6,610 rejected)	Generation	Groq (see section 10)

ELEVATOR PITCH

The one sentence to open with. This is not a chatbot with a legal flavour. It is a closed-book retrieval system: it answers only from statute text and judgments it actually retrieved, cites them, and **abstains when the corpus cannot support an answer**. Every design decision below was chosen by measurement, and the measurements are in the repository.

1. What problem this solves

A general-purpose LLM asked "what is the punishment under IPC 302?" will produce a confident answer whether or not it knows. In law that failure mode is unacceptable: a wrong section number or an invented citation is worse than no answer. Two further complications are specific to Indian criminal law:

- The **Indian Penal Code, 1860 was repealed on 1 July 2024** and replaced by the Bharatiya Nyaya Sanhita, 2023. Both remain legally relevant - offences before that date are still tried under the IPC - so the system must carry both and never silently blend them.
- Statutes state the rule; **judgments state how the rule is applied**. Answering well needs both, and they need different handling: a statutory section is a self-contained unit, a judgment is tens of thousands of characters of prose.

The system therefore keeps statutes and judgments in separate collections, labels every provision with its legal status, and refuses to answer when retrieval is too weak to support one.

2. System architecture

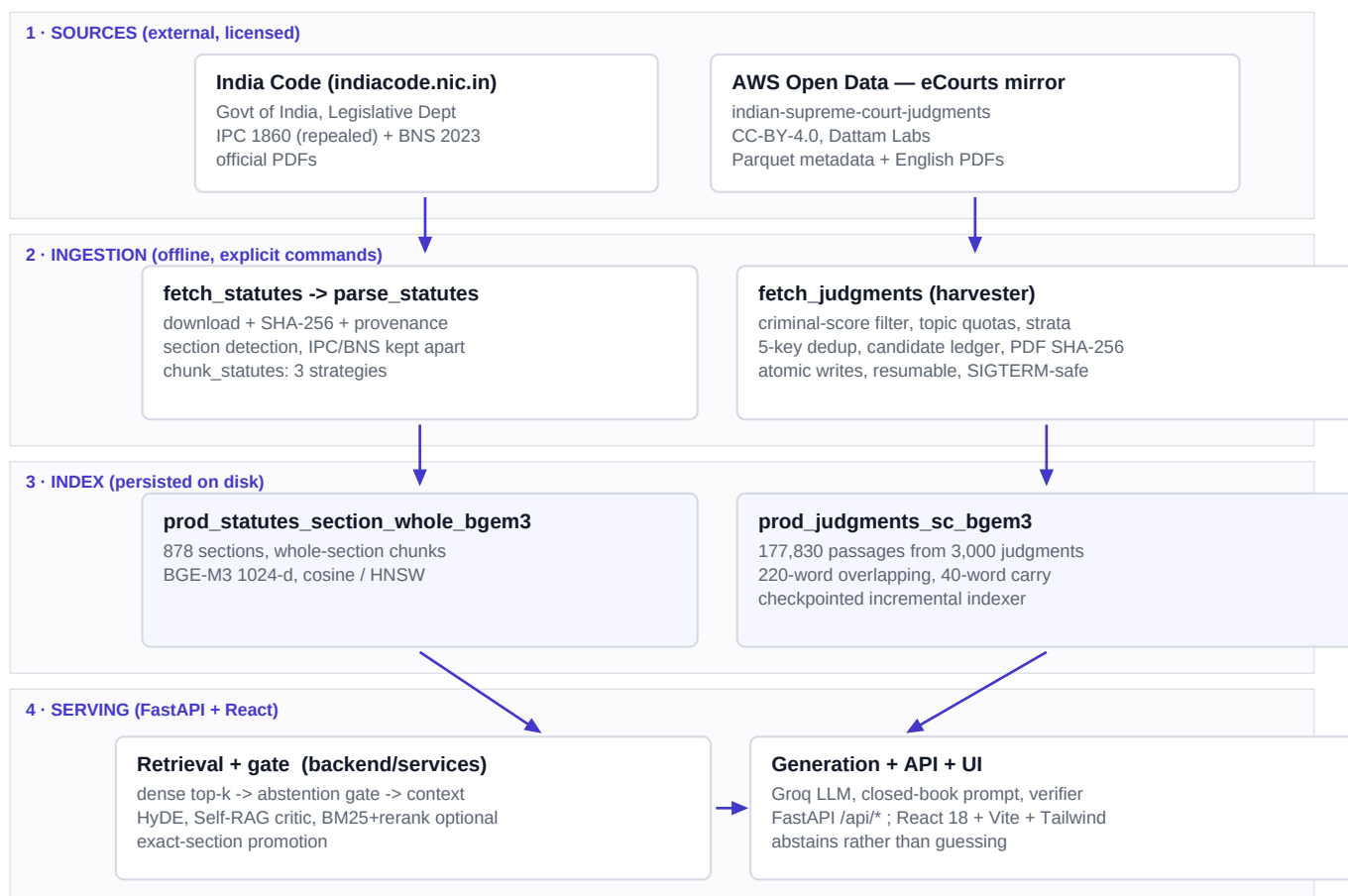


Figure 1. Four layers. Ingestion is offline and explicitly invoked - starting the web app never triggers a download, a re-index, or any model training. The backend loads the already-persisted ChromaDB, and the embedding model is loaded lazily on the first query, so an idle app does no ML work at all.

3. Where the data comes from

	Statutes	Judgments
Source	India Code (indiacode.nic.in), Legislative Department, Ministry of Law and Justice	indian-supreme-court-judgments on the AWS Registry of Open Data - a mirror of the official eCourts portal (judgments.ecourts.gov.in)
Licence	Government of India public documents	CC-BY-4.0 , maintained by Dattam Labs
Coverage	IPC 1860 (repealed 2024-07-01), BNS 2023 (in force)	Supreme Court of India, 1950-present
Format	Official PDFs	Year-partitioned Parquet metadata + English PDFs
Why this source	Authoritative government publisher; no scraping	Explicit open licence and bulk access, unlike scraping sci.gov.in. Every record keeps the official citation and neutral citation, so any judgment can be checked against the court's own portal.

NOTHING IS INVENTED

Provenance rule, worth stating explicitly to a mentor. Case name, petitioner, respondent, court, date, citation, neutral citation, judge and disposal nature are copied *verbatim* from the published metadata. Statutes referred, sections referred and topics are *extracted from the judgment text*. Nothing anywhere in the corpus is inferred, guessed, or generated by a model.

4. The ingestion pipeline

4.1 Statutes

`fetch_statutes` downloads each Act once per run, records SHA-256, byte size, publisher and retrieval date, then `parse_statutes` detects section boundaries and `chunk_statutes` produces chunks. IPC and BNS are never merged: each section carries `legal_status` (*repealed* / *in_force*) plus commencement and repeal dates, and retrieval always surfaces that status so a reader is never shown a repealed provision without knowing it.

4.2 Judgments - a filtered corpus, not a sample

The dataset holds roughly 44,000 Supreme Court judgments across 77 years. Most are tax, service or constitutional matters. Taking them wholesale would bury criminal-law signal in noise, so each candidate is scored on the statutes and vocabulary present in its text and admitted in two tiers:

Gate	Rule	Purpose
Base - length	at least 3,000 characters	below this it is an order sheet or a failed extraction, not a judgment
Base - relevance	<code>criminal_score</code> $\geq$ 6	4 points per core statute (IPC/BNS/CrPC/BNSS), 2 for Evidence/NDPS/POCSO, plus capped vocabulary density
General fill	<code>criminal_score</code> $\geq$ 10	the bar for bulk volume
Quota rescue	score 6-9 and covers a topic below its floor	keeps thin topics reachable without lowering the bar generally
Topic ceiling	rejected only if every topic it covers is at its cap	murder and bail dominate reported appeals; caps are 50% of target

Result: **9,350 candidates examined, 2,740 retained (29.3%), 6,610 rejected** - 5,967 of those as not criminal-law. Every rejection is recorded with its reason in a candidate ledger, so the shape of the corpus is auditable rather than taken on trust, and a re-run never re-downloads a known reject.

4.3 Engineering safeguards in the harvester

Safeguard	What it does and why it exists
Merge-on-load	The first version started from an empty list and overwrote the corpus file, so a second run destroyed the first run's work . It now loads what exists and appends.
Five-key dedup	source URL, (year, PDF path), neutral citation, official citation, and SHA-256 of the extracted text. The first four are checked before downloading, so a duplicate costs no bandwidth. This is not theoretical: the source dataset itself repeats rows - 8,799 rows share a <code>case_id</code> .
Required-metadata gate	A judgment with no neutral citation cannot be cited back to the court, so it is skipped pre-download rather than ingested unusable.
Retained PDFs + checksums	Every kept PDF is stored with its own SHA-256, so text extraction is reproducible and re-verifiable offline without re-fetching.
Atomic writes	Corpus, ledger and provenance are written temp-file-then-rename. An interrupted run can never leave a half-written corpus.
Graceful stop	SIGTERM finishes the candidate in flight, saves, and exits. Re-running continues where it stopped.
Persisted stratum accounting	Allocation is derived from the records on disk, never from a counter in the running process. See section 12 - this one is a real bug story.

5. Chunking, embeddings and storage

5.1 Why statutes and judgments are chunked differently

A statutory section is a self-contained legal unit, so it is stored **whole**. A judgment has no equivalent unit, so it is split into **220-word overlapping passages with a 40-word carry**, sentence-aligned, each stamped with a compact case header (case name, court, date) so a retrieved paragraph is always attributable. The overlap exists so a holding split across a boundary is still retrievable from either side.

5.2 Which model, and the evidence for it

Three variables were tested against a 43-query evaluation set with 57 gold labels, holding everything else constant. These are measured numbers from `evaluation/results/`:

Cfg	Configuration	Model	Chunking	R@5	MRR	p50 ms
A	Dense, fixed-window chunks	bge-base	fixed_window	0.346	0.310	37
B	Dense + whole-section chunks	bge-m3	section_whole	0.763	0.633	64
B	Dense + whole-section chunks	bge-base	section_whole	0.662	0.602	34
C	Hybrid BM25 + dense (RRF)	bge-m3	section_whole	0.702	0.666	65
D	Hybrid + cross-encoder rerank	bge-m3	section_whole	0.697	0.669	627

- **Chunking mattered most.** Whole-section beat fixed-window: MRR 0.310 -> 0.602, a 94% gain.
- **bge-m3 beat bge-base** on the same chunking: R@5 0.763 vs 0.662, +15%.
- **Reranking was rejected on evidence.** Config D scores higher MRR (0.669 vs 0.633) but *lower* Recall@5 (0.697 vs 0.763) and costs roughly **10x the latency** (627 ms vs 64 ms p50). For a system that feeds five chunks to an LLM, recall matters more than rank order, so production runs dense-only. Hybrid and reranking remain implemented and reproducible.

5.3 ChromaDB, and where FAISS fits

Store	Role	Detail
ChromaDB	Production vector store	Persistent client on local disk. HNSW index, cosine space, unit-normalised vectors. Two production collections kept deliberately separate.
FAISS	Implemented alternative / legacy path	<code>faiss-cpu</code> with a flat index is wired in (<code>modules/faiss_index.py</code> , <code>VECTOR_STORE</code> switch in config). Not the production path.
BM25	Lexical arm of hybrid retrieval	<code>rank-bm25</code> , fused with dense results by Reciprocal Rank Fusion (<code>k=60</code>).
Knowledge graph	Citation / lineage features	NetworkX graph persisted as a pickle, powering the case-lineage and clusters endpoints.

A REAL BUG THIS NAMING PREVENTS

Why the collection name encodes the model. A Chroma collection is only valid for the dimensionality it was built with - `bge-base` is 768-d and `bge-m3` is 1024-d. Querying one with the other's vectors is the exact failure this project started with: the config declared `bge-m3` while the index on disk was 768-d, so vector search silently returned nothing. Hence `prod_statutes_section_whole_bgem3`.

6. Retrieval and the abstention gate

The request path for a statute question:

query → (optional HyDE expansion) → dense search, CANDIDATE_K = 15 → exact-section promotion → abstention gate → TOP_K = 5 to the LLM → closed-book generation → grounding verifier → cited answer or refusal

Component	What it does
HyDE	The LLM drafts a hypothetical answer, which is embedded and used as the query. Helps when the user's wording shares no vocabulary with the statute.
Exact-section promotion	If the user names a section ("IPC 302"), that section is promoted into the candidate set directly. Pure vector similarity can rank the literally-requested provision below a semantically similar neighbour.
Abstention gate	Multi-signal: a hard similarity floor (0.40), a soft floor (0.486), a minimum number of supporting candidates, and a minimum margin between the top hits. Fails closed .
Self-RAG critic	A second LLM pass checks the drafted answer against the retrieved evidence and can trigger a retry.
Grounding verifier	Checks that every section the answer cites was actually in the retrieved set.
Parent-document retrieval	Search over 256-token children, return the 512-token parent for context.

THE HONEST CAVEAT

Be ready for this question: "how do you know the gate works?" On the evaluation set it scores perfectly - 38/38 answerable questions answered, 5/5 unanswerable correctly refused, zero false abstentions. **Do not present that as a generalisation.** The thresholds were chosen by inspecting similarity ranges on *that same set*, so it is a fit to the data, not an estimate of held-out performance - and there are only 5 unanswerable queries. The repository states this limitation in the results file itself. Saying so first is much stronger than being caught on it.

7. Generation layer

The LLM is called through a pluggable backend (RAG_LLM_BACKEND: groq | openai | gemini | ollama). The current default is **Groq** running openai/gpt-oss-120b, with a fallback model list and SSE streaming. The API key is read from .env, which is gitignored. Context is capped at 5 chunks / 3,000 tokens.

The prompt is a **closed-book contract**: answer only from the supplied provisions, cite them, and say so explicitly when the corpus cannot support an answer. This contract is what turns retrieval into a safety property rather than a convenience.

7.1 The A-E generation experiment (built, not yet run)

A five-configuration ablation, where each config answers the *same* questions so any difference is attributable to the one safeguard added:

Cfg	Name	What it adds
A	baseline	LLM alone, no retrieval - the control. Shows what the model produces with no evidence, which is the behaviour the rest of the stack exists to prevent.
B	standard_rag	retrieval + a permissive prompt - ordinary RAG
C	closed_book	adds the closed-book contract
D	closed_book_gate	adds the fail-closed retrieval gate

Cfg	Name	What it adds
E	full	adds the grounding verifier and retry

WORK IN PROGRESS - SAY THIS UP FRONT

Status, stated plainly. The harness is complete and tested but the comparison has **not been run to completion**.

Without an API key it refuses to emit output rather than estimating - every number it produces must come from a real model call. Note also that `factuaL_correctness` is left null by design: scoring it requires a qualified human reader, so raw answers are saved per question for later marking rather than being auto-scored.

8. Backend

Layer	Choice	Notes
Framework	FastAPI	async, automatic OpenAPI docs at /docs
Server	Uvicorn	bound to 127.0.0.1:8000 - loopback only, not exposed
Streaming	sse-starlette	token streaming to the UI
Auth	JWT	/auth/register, /login, /me
Rate limiting	slowapi	implemented, currently disabled for local speed
Caching	diskcache / Redis	24-hour TTL query cache
Background work	RQ + Redis, watchdog	delta worker for incremental document ingest
PDF handling	PyMuPDF, unstructured, docling, pytesseract	layout-aware extraction with OCR fallback

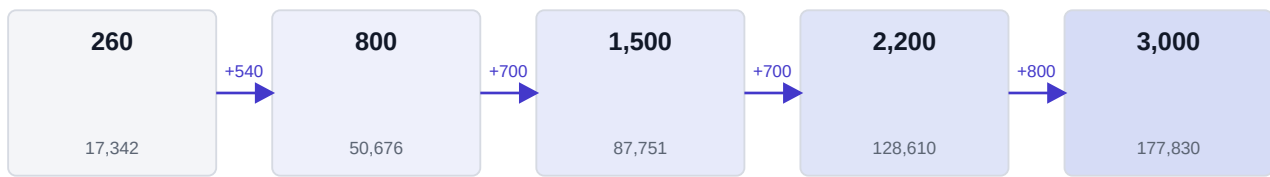
Router (all under /api except auth): statute (statute QA), query (retrieve / answer / stream), index (corpus + build status), graph (clusters, case lineage), compare, summary, critique, export, eval.

9. Frontend

Concern	Library
Framework / build	React 18 + TypeScript 5, Vite 5
Styling	Tailwind CSS 3, tailwind-merge, clsx
Components	Radix UI (select, tabs, tooltip), lucide-react icons
Server state	TanStack Query 5; axios for HTTP
Client state	Zustand
Routing	React Router 6
Visualisation	D3 7 and React Flow 11 (knowledge graph, timelines)
Markdown	react-markdown + remark-gfm (renders cited answers)
Motion	Framer Motion 11

Nine workspace views: Statute QA, Case Law Chat, Sources, Case Summary, Compare Timeline, Conflicts, Critique, Knowledge Graph, and case management. The Statute QA view exposes an IPC / BNS / Both / Auto selector so the user controls which code is searched - the repealed-vs-in-force distinction is surfaced in the interface, not hidden.

10. Corpus growth and incremental indexing



Every expansion appends. Existing passage IDs never change, and the indexer verifies the stored prefix byte-for-byte before embedding anything new.

Cumulative MPS time saved by not rebuilding: about 18 hours.

Figure 2. The corpus was built in four stages. Because judgments are only ever *appended*, passage IDs of existing material never shift, so their embeddings stay valid.

The indexer does not take that on trust. On startup it reads the IDs already in ChromaDB, treats a slice of 256 passages as complete only when every ID is present **and** the stored documents match the freshly built chunks, and embeds only the remainder. Progress is checkpointed after every slice, so an interruption costs at most one slice - roughly a minute - not the run.

Expansion	Judgments	Passages embedded	Passages reused	MPS time
260 (initial build)	260	17,342	0	~1 h 12 m
-> 800	+540	33,334	17,152	~2 h 07 m
-> 1,500	+700	37,319	50,432	~2 h 18 m
-> 2,200	+700	41,058	87,552	2 h 26 m
-> 3,000	+800	49,318	128,512	5 h 58 m

Throughput is a measured **4.4-4.7 passages/second** for BGE-M3 on Apple Silicon MPS. Rebuilding from scratch at 3,000 judgments would take about 11 h 30 m; the incremental path took 5 h 58 m.

11. Corpus design - how 2,200 was chosen, not just reached

Each expansion was allocated across **strata** - year bands chosen to close a measured gap - rather than harvested indiscriminately. The final round:

Stratum	Years	Allocated	Retained	Why that band
pocso_ndps_hunt	2018-2023	340	340	POCSO is 0% before 2018 and 5.7% here; NDPS 8.5%
bns_exhaust	2024-2026	200	200	the only band where BNS/BNSS judgments exist
mid_2001_2017	2001-2017	70	70	highest acceptance rate measured (40.3%)
historical_fill	1974-1999	60	60	22 years whose metadata had never been fetched
deepen_1950_1972	1950-1972	30	30	early IPC jurisprudence

11.1 Topic coverage - all 21 floors met

Topic	Count	Topic	Count	Topic	Count
murder	877	conspiracy	444	dying declaration	123
bail	820	quashing	354	UAPA	100
common intention	674	circumstantial ev.	323	BNS / BNSS / BSA	66
sentencing	556	attempt	304	POCSO	63
sexual offences	269	theft / robbery	299	juvenile justice	57
corruption	246	confession / recovery	208	PMLA	50
cheating	250	dowry / cruelty	193	NDPS	141

EXPECT THIS QUESTION

The BNS answer, if asked why coverage is only 66. It is **source-limited, not effort-limited**. Measured density of new-codes judgments: 3.9% (2024), 6.2% (2025), **33.3% (2026)** - but only about 40 qualifying 2026 candidates existed in the entire dataset. Most 2024-25 offences predate the 1 July 2024 commencement, so reported judgments applying the BNS simply do not exist in volume yet. The measured ceiling from this source is roughly 82, and reaching it would have consumed 82% of an expansion budget at POCSO's expense. The floor was set to what is achievable rather than to a round number.

12. A bug worth telling them about

Stratum progress was originally a counter **local to the running process**. A harvest that was interrupted and resumed restarted every stratum's allocation at zero. During the 800-judgment build the harvester was stopped and restarted three times, and one stratum reached **202 judgments against an approved 120**.

The fix: allocation is now derived from the `stratum` field on the persisted records, never from process state. It is pinned by **14 regression tests** including simulated runs of up to 25 interruptions, and the old logic was replayed to confirm the tests actually catch it - it reaches 360 against a target of 120 where the new planner stays at 150/150.

This is a good story to volunteer: it shows the difference between code that works when nothing goes wrong and code that holds under interruption.

13. Verification

A **21-check readiness suite** runs before and after every expansion, with zero network access. It proves, among other things:

- the previous corpus is **byte-identical** to a frozen snapshot (SHA-256 pinned in a manifest)
- all previously-embedded passages match the rebuilt chunks **byte-for-byte** - all 87,751 compared, not sampled
- new passage IDs append after the existing index; zero prefix mismatches
- every record has case name, citation, neutral citation, date, judge, source URL - 2,200/2,200
- unique on URL, neutral citation, official citation and text hash - 2,200/2,200 on each
- duplicates present in the source metadata are caught - 4,412 of 4,412 flagged
- the ledger blocks re-fetching all 4,715 previously-rejected candidates
- no stratum exceeds its allocation, under 25 simulated interruptions

Alongside it, **206 unit tests** cover chunking, IR metrics, the grounding verifier, the closed-book contract, preprocessor caching and the generation harness.

14. Limitations - know these before you are asked

Limitation	The honest position
Abstention thresholds are fitted	Tuned on the same 43-query set they are reported on, with only 5 unanswerable queries. Not a generalisation claim. A held-out set is the obvious next step.
Evaluation set is small	43 queries, 57 gold labels, single-annotator. Enough to rank configurations against each other, not to certify absolute accuracy.
A-E comparison not completed	The harness is built and tested; the run needs API budget. No numbers are claimed from it.
Factual correctness unscored	Deliberately null - it needs a qualified human reader. Raw answers are stored per question.
Judgment retrieval not benchmarked like statutes	The 43-query set targets statutes, where a question has an unambiguous correct section. Judgment relevance has no comparable ground truth yet.
Supreme Court only	No High Court or trial court judgments. Vertical coverage, not exhaustive coverage.
Section extraction is regex-based	v2 attaches the nearest statute within 120 characters and will not cross a sentence boundary. Better than v1 - whose top 'citations' were 3, 4, 5 and 2 from prose - but not a parser.
Not legal advice	Surfaced in the UI. The IPC copy indexed reflects amendments only up to 1997.

15. Questions a mentor is likely to ask

Why RAG instead of fine-tuning a model on legal text?

Three reasons. **Citability** - RAG can point at the provision it used; a fine-tuned model cannot. **Currency** - the BNS replaced the IPC in July 2024; updating a corpus is a re-index, updating a fine-tune is a retraining run. **Abstention** - a retrieval system can detect that it found nothing and refuse. A fine-tuned model has no equivalent signal.

Why ChromaDB rather than FAISS, Pinecone or pgvector?

Chroma persists metadata alongside vectors, which this project needs heavily - legal status, citation, court, date, section number all travel with the chunk and are used for filtering and display. It runs embedded with no server to operate, which suits a local capstone. FAISS is implemented as an alternative and is faster on pure vector search, but stores no metadata, so it would need a second store beside it. Pinecone is hosted, which adds cost and a network dependency for no benefit at this scale.

Why bge-m3 over OpenAI embeddings?

It is open-weights and runs locally, so there is no per-query cost or data egress - relevant for legal text. It is multilingual (100+ languages, including Hindi, Kannada and Tamil), which matters for Indian legal material. And it was measured to beat bge-base by 15% on Recall@5 on this corpus.

Your reranker scored a higher MRR. Why is it not in production?

Because it scored *lower Recall@5* - 0.697 vs 0.763 - at roughly 10x the latency. The system passes five chunks to an LLM, so what matters is whether the right provision is in those five at all, not whether it is ranked first. Optimising MRR here would have cost real answers. It remains available behind a flag and is fully reproducible.

How do you stop it from hallucinating a section number?

Four layers, and the honest answer is that none alone is sufficient. Retrieval supplies the only permitted evidence; the closed-book prompt forbids outside knowledge; the abstention gate refuses when similarity, support count or margin is too low; and the grounding verifier checks that every cited section was actually retrieved. The A-E experiment exists precisely to measure how much each layer contributes - and it has not been run yet.

How do you handle the IPC being repealed?

They are never merged. Each section carries legal_status, commencement date, repeal date and what replaced it. Retrieval always surfaces that status, and the UI lets the user search IPC, BNS, both, or let the system infer. A prosecution begun under the IPC continues under it, so the repealed code is retained rather than deleted.

How would you scale this to High Courts, or to 50,000 judgments?

The ingestion side already scales - it is incremental, deduplicated, ledgered and resumable. The constraints are embedding throughput (about 4.5 passages/s on this laptop; 50,000 judgments would be days of MPS time, so a GPU or a batch service) and ChromaDB, which is embedded and single-node - at that scale a server-based store with proper ANN tuning would be needed. Storage grows about 21 KB per passage.

What is the single weakest part of this project?

The evaluation set. 43 queries with one annotator is enough to compare configurations against each other, which is what it was used for, but not enough to make an absolute accuracy claim - and the abstention thresholds are fitted to it. Expanding to a held-out, multi-annotator set is the highest-value next step.

16. How to run it

```
./scripts/start.sh # backend :8000 + frontend :5173
./scripts/status.sh # what is running
./scripts/stop.sh # stop both
```

```
python -m backend.ingestion.fetch_judgments --target 2200 # harvest (additive)
python -m backend.ingestion.verify_expansion_readiness # 21 integrity checks
./scripts/index-judgments.sh # incremental embedding
./scripts/index-status.sh / ./scripts/index-stop.sh # progress / safe stop
python -m backend.ingestion.corpus_stats # corpus report
```

DESIGN PRINCIPLE

Starting the web app never downloads, re-indexes or trains anything. Every heavy job is a separate, explicit command, and the embedding model loads lazily on the first query - so an idle app does no ML work at all. That separation is deliberate and worth pointing out.

Repository layout: backend/ingestion/ (harvest, parse, chunk, index), backend/routers/ (API), backend/services/ (retrieval, abstention, generation), modules/ (embedder, BM25, FAISS, reranker, graph), evaluation/ (harnesses + measured results), frontend/src/ (React app), docs/ (architecture, data pipeline, corpus selection, evaluation), scripts/ (process management).